# Photo Duel — Design Doc

Oct 23, 2026 · @Ambika Eshwar

## Overview & goals

Photo Duel is an Android app that groups near-duplicate photos from the camera roll and runs quick head-to-head comparisons until the user has picked a favorite in each group. It replaces the slow loop of scrolling back and forth through a burst trying to remember which shot was best.

**Target user:** someone who takes several versions of every shot (portraits, group photos, food, scenery) and wants to send or keep only the best one.

**Goals**

- Automatically find groups of similar shots in the camera roll, with no manual album building.
- Find the favorite in a group of n photos in at most n − 1 decisions.
- Make tiny differences (a blink, a half-smile, motion blur) obvious through in-place hold-to-flip comparison on aligned images.
- Act on the result: favorite or share the winner, and move the rest to trash in one step.
- Run fully on-device. No account, no cloud upload, no network permission needed for core features.

**Non-goals (v1)**

- Editing, filters or cropping.
- Cloud sync or multi-device state.
- Video and Live/motion photos (stills only).
- A full ranking of every photo; v1 cares about the top one, optionally a runner-up.

## User flow

The core loop is: open the app, pick a group, duel until one photo remains, then act on the winner.

1. **First launch:** request media permission and explain why. Index the camera roll in the background, newest first, with a progress indicator. Groups become usable as soon as they are found.
2. **Groups screen:** a vertical feed of unresolved groups. Each card shows a thumbnail strip, photo count and date. Resolved groups move to a "Done" tab. Default scope is the last 30 days, with a toggle for all time.
3. **Duel screen:** one photo fills the screen. The current favorite (the "king") is shown by default; holding a button or the image shows the challenger in the same spot. The user picks one; the winner stays and the next challenger loads.
4. **Result screen:** the winner is shown large with actions: Share, Favorite, Keep runner-up, and "Trash the rest" (with a count). The user can also skip actions and just mark the group done.
5. **Group editing (any time):** remove a photo from a group, merge two adjacent groups, or split a group at a chosen photo.

New photos are picked up on app open and through a MediaStore content observer while the app runs.

## Grouping

Groups are built in two passes: split the camera roll into time windows, then cluster each window by visual similarity. Only groups of two or more photos are shown.

**Pass 1 — time sessions.** Sort photos by `DATE_TAKEN` (fall back to `DATE_ADDED`). Start a new session whenever the gap between consecutive photos exceeds a threshold (default 60 s, configurable). This keeps comparisons cheap and stops a photo from today being grouped with one from last year.

**Pass 2 — visual clustering within a session.**

- Compute an image embedding for each photo on-device. Recommended: a small image-embedding model run through MediaPipe Image Embedder or TFLite (e.g. MobileNet-V3 or a quantized CLIP image encoder). Embed a downscaled copy (around 224 px).
- Compute cosine similarity between photos in the session and run agglomerative clustering (average linkage) with a similarity threshold (start at 0.85, tune on real data).
- Also compute a 64-bit perceptual hash (dHash) as a cheap pre-filter and a tiebreaker for exact or near-exact duplicates.

- Exclude screenshots and other non-camera images by filtering on `RELATIVE_PATH` (DCIM/Camera and similar) by default.

**Caching.** Store each photo's embedding and hash in the database, keyed by MediaStore ID plus `DATE_MODIFIED`, so indexing is incremental. Run indexing in a WorkManager job that is resumable and respects battery-low constraints.

**Manual corrections.** User edits (remove, merge, split) are stored as overrides and are never undone by a later re-cluster. A photo removed from a group stays out of it.

## Comparison & ranking

Ranking uses a king-of-the-hill format: the current favorite stays on screen and each other photo challenges it once. This finds the winner in exactly n − 1 decisions.

**Ordering.** The first king is the highest-scoring photo by a heuristic pre-score (below), so strong candidates meet early. Challengers are ordered by pre-score, descending.

**Pre-score (v1 heuristic).** Computed on-device with ML Kit Face Detection (classification mode on) plus a sharpness measure:

- Mean eyes-open probability across detected faces.
- Mean smiling probability.
- Sharpness: variance of the Laplacian on the face regions (or the whole image if no faces).

Combine as a weighted sum; weights live in one config object so they are easy to tune. The pre-score only orders the duel. It never decides the winner.

**Runner-up (optional).** After the winner is chosen, offer "Pick a runner-up?". Run one more king-of-the-hill among the photos that lost directly to the winner (they are the strongest remaining candidates).

**Hold-to-flip comparison.**

- The king is shown by default. Pressing and holding (on the image or a large bottom button) shows the challenger in the same place. Releasing shows the king again.
- Flips are instant: both full-resolution bitmaps are decoded and held in memory before the duel step is shown. Prefetch the next challenger in the background.
- A small label ("Current pick" / "Challenger") and a colored edge make it clear which one is showing.

**Alignment (required for flip to work).** Handheld bursts drift by a few pixels, which makes the whole frame jitter on flip and hides the real differences. Before showing a challenger, register it to the king:

- Detect ORB features on downscaled grayscale copies, match with brute-force Hamming plus ratio test, estimate a homography with RANSAC (OpenCV).
- If enough inliers are found and the transform is close to a small translation/rotation/scale, warp the challenger into the king's frame for display. Otherwise show it unaligned (better than a wrong warp).
- Cache the transform per (king, challenger) pair. Never modify the original files.

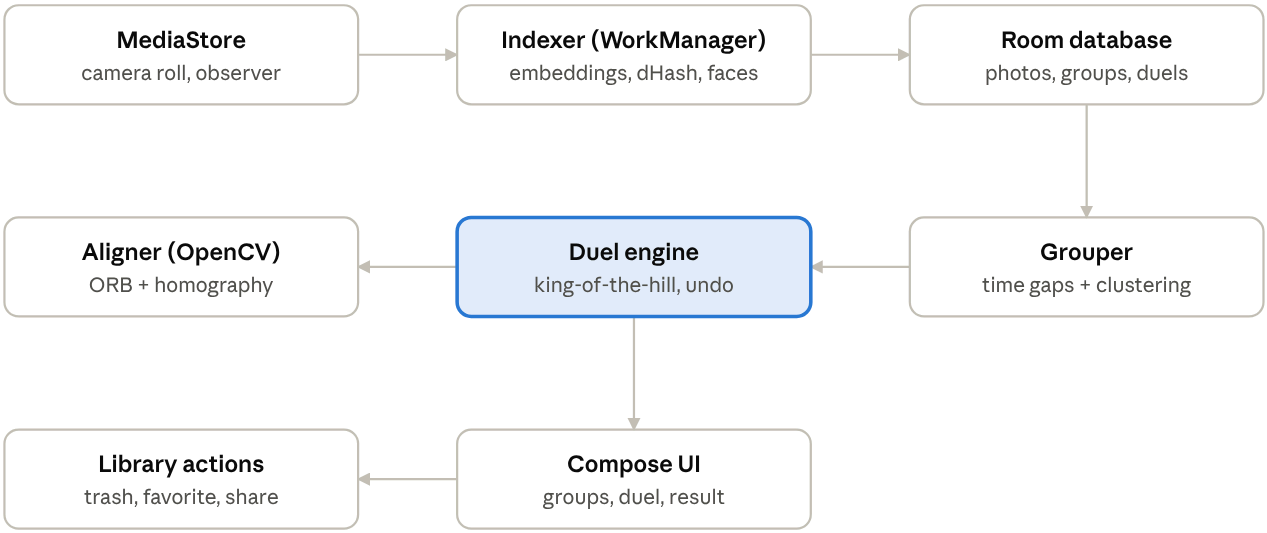
**Zoom and pan.** Pinch-zoom and pan state is shared between the two images, so the user can zoom into a face and flip there. A double-tap zooms to the largest detected face; double-tap again returns to fit.

**Picking.** Swipe up keeps the challenger (it becomes king); swipe down keeps the king. Two labeled buttons do the same for discoverability. Undo reverts the last decision. Every decision is logged (see Data model) so it can later train a personal preference model.

## Architecture & tech stack

A single-activity Kotlin app with Jetpack Compose, split into a background indexing pipeline and a foreground duel UI that share one Room database.

**Indexing runs in the background; duels read from the database**




app architecture · 8 components

The indexer and grouper never block the UI; the duel engine only reads groups that are already built, and writes each decision back to the database.

| Area | Choice |
|---|---|
| Language / UI | Kotlin, Jetpack Compose, Material 3, single activity + Navigation Compose |
| Min / target SDK | minSdk 30, target the latest stable SDK |
| Architecture | MVVM with ViewModels + StateFlow, Hilt for DI, coroutines |
| Library access | MediaStore queries + ContentObserver |
| Image loading | Coil for thumbnails; BitmapFactory/ImageDecoder for full-res duel bitmaps |
| Embeddings | MediaPipe Image Embedder (or TFLite) with a bundled model |
| Faces | ML Kit Face Detection (bundled model, classification on) |
| Alignment / sharpness | OpenCV Android SDK (ORB, findHomography, warpPerspective, Laplacian) |
| Storage | Room (SQLite) |
| Background work | WorkManager |

**Gradle modules:** `app` (navigation, DI), `core-data` (Room, MediaStore repository), `core-ml` (embedder, face scoring, sharpness), `feature-groups`, `feature-duel` (duel engine, aligner, flip view).

## Data model

Five Room tables hold everything; photo files themselves are never copied, only referenced by MediaStore ID.

| Table | Key columns | Notes |
|---|---|---|
| `photo` | `media_id` (PK), `date_taken`, `date_modified`, `width`, `height`, `embedding` (BLOB, float32), `dhash` (Long), `pre_score` (Float), `indexed_at` | Re-index when `date_modified` changes; delete row when the media is gone |
| `photo_group` | `group_id` (PK), `created_at`, `status` (UNRESOLVED / DONE / DISMISSED), `winner_media_id`, `runner_up_media_id` | One row per detected group |
| `group_member` | `group_id`, `media_id`, `position` | Ordered by capture time |
| `group_override` | `id`, `kind` (EXCLUDE / MERGE / SPLIT), `media_ids` (JSON), `created_at` | Applied after every re-cluster so user edits stick |
| `duel_decision` | `id`, `group_id`, `winner_media_id`, `loser_media_id`, `decided_at`, `flip_count`, `undone` (Boolean) | Training data for a future preference model |

Grouping is deterministic given the same photos, thresholds and overrides, so groups can be rebuilt from `photo` + `group_override` at any time.

## Library actions & permissions

All changes to the user's library go through MediaStore system requests, so Android shows its own confirmation and the app never deletes files directly. Losers go to the system trash, not permanent deletion.

| Action | API | Notes |
|---|---|---|
| Trash the rest | `MediaStore.createTrashRequest` | Batch all losers into one request, one confirmation dialog; restorable from the gallery's trash |
| Favorite winner | `MediaStore.createFavoriteRequest` | Shows up as a favorite in Google Photos and other galleries |

| Action | API | Notes |
|---|---|---|
| Share winner | `Intent.ACTION_SEND` with the content URI | Standard share sheet |
| Open in gallery | `Intent.ACTION_VIEW` | For edits outside the app |

**Permissions**

- Android 13+: `READ_MEDIA_IMAGES`. Android 14+: also handle partial access (`READ_MEDIA_VISUAL_USER_SELECTED`) by working on whatever the user selected and offering a button to grant full access.
- Android 11–12: `READ_EXTERNAL_STORAGE`.
- No `INTERNET` permission in v1.

These trash and favorite APIs need Android 11 (API 30), so **minSdk is 30**.

## Milestones, testing & open questions

Build in five milestones, each ending in something runnable on a real phone. The duel screen comes before smart grouping because it is the part most worth validating by hand.

1. **M1 — Skeleton:** project setup, permissions, MediaStore repository, grouping by time gap only, groups list with thumbnails.
2. **M2 — Duel:** king-of-the-hill engine, hold-to-flip view with prefetched full-res bitmaps, shared zoom/pan, swipe and button picking, undo, result screen with Share.
3. **M3 — Alignment:** OpenCV ORB + homography aligner with fallback to unaligned, transform cache.
4. **M4 — Smart grouping:** on-device embeddings, dHash, clustering within sessions, incremental WorkManager indexing, manual remove/merge/split with overrides.
5. **M5 — Polish:** pre-score ordering with ML Kit faces and sharpness, double-tap to face, runner-up pass, trash and favorite requests, Done tab, settings for thresholds.

**Testing**

- Unit tests: duel engine ($n - 1$ decisions, undo, runner-up candidate set), time-session splitting, clustering on fixture embeddings, override application.
- Instrumented tests: Room DAOs and migrations; MediaStore repository against a seeded emulator gallery.

- Aligner tests: synthetic shifted/rotated copies of fixture images must recover the transform within 2 px; unrelated images must fall back to unaligned.
- Performance targets on a mid-range device: flip latency under 16 ms (no decode on flip), challenger ready within 300 ms of a pick, indexing at least 5 photos/s.
- Manual test set: a folder of real bursts (portraits, group shots, food, landscapes) used to tune the similarity threshold.

### Open questions

- Default time gap (60 s) and similarity threshold (0.85) need tuning on real camera rolls.
- Which embedding model gives the best grouping per MB of APK size?
- Should the app read RAW+JPEG pairs or Google Camera burst folders specially?
- Future: train a small personal preference model on `duel_decision` to replace the heuristic pre-score.